



# 链表的高级主题选讲

- 静态链表
- 跳跃表
- 侵入式链表

数据之法  
结构之美  
算法之道





# 链表的高级主题选讲

- 静态链表
- 跳跃表
- 侵入式链表

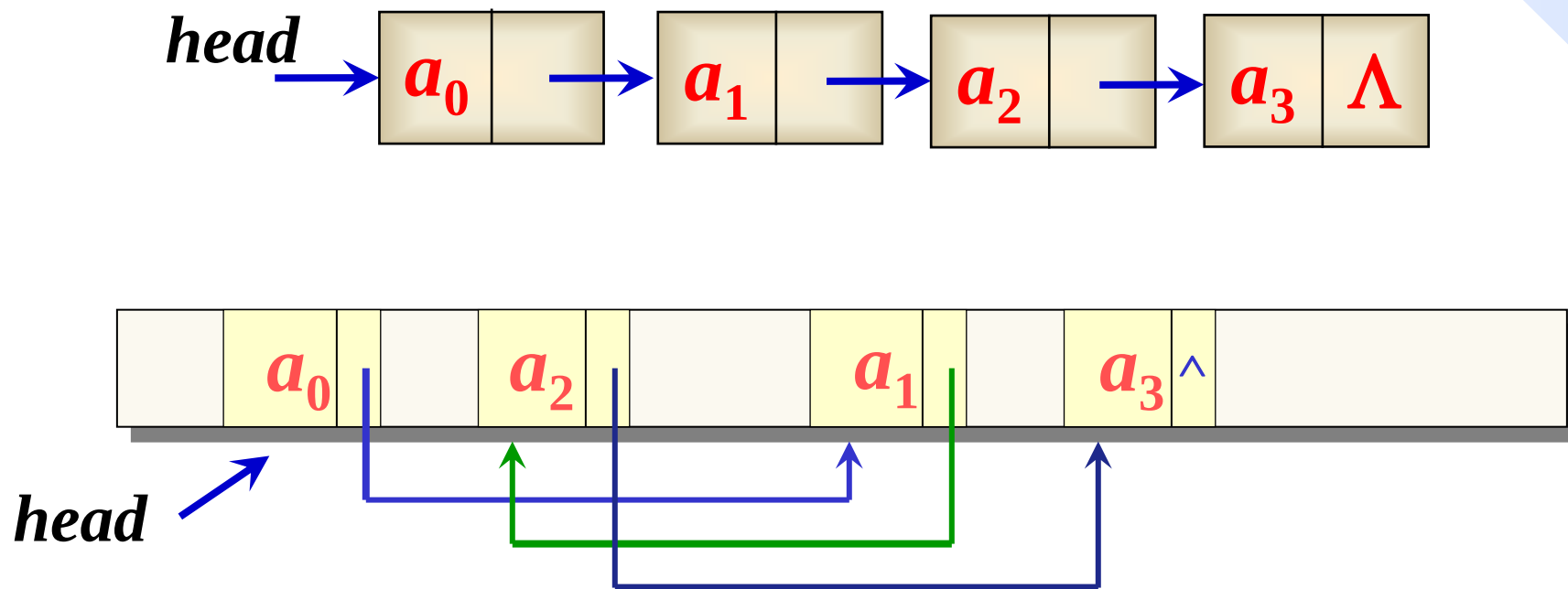
数据之法  
结构之美  
算法之道



# 静态链表

有些程序设计语言没有指针类型，如何实现链表？

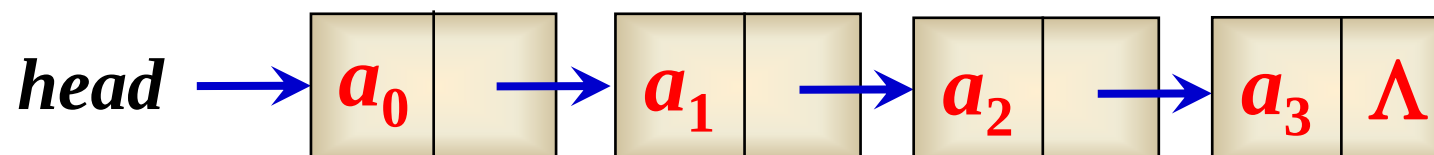
回顾链表在内存的存储方式



# 静态链表

```
int data[12];
int next[12];
```

|      | 0 | 1     | 2 | 3     | 4 | 5 | 6     | 7 | 8 | 9 | 10    | 11 |
|------|---|-------|---|-------|---|---|-------|---|---|---|-------|----|
| data |   | $a_0$ |   | $a_2$ |   |   | $a_1$ |   |   |   | $a_3$ |    |
| next |   |       |   |       |   |   |       |   |   |   |       |    |

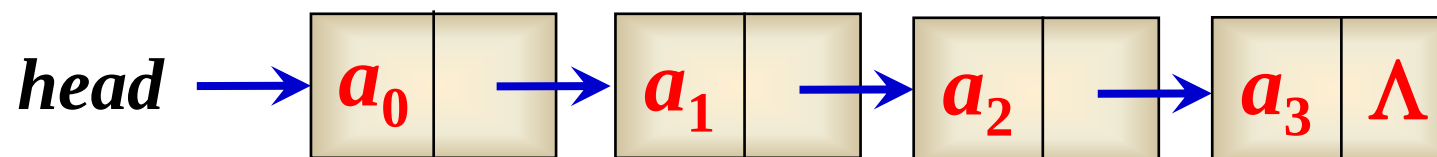


用数组存储链表的元素  
用数组的下标模拟指针

# 静态链表

```
int data[12];
int next[12];
```

|      | 0 | 1     | 2 | 3     | 4 | 5 | 6     | 7 | 8 | 9 | 10    | 11 |
|------|---|-------|---|-------|---|---|-------|---|---|---|-------|----|
| data |   | $a_0$ |   | $a_2$ |   |   | $a_1$ |   |   |   | $a_3$ |    |
| next |   | 6     |   | 10    |   |   | 3     |   |   |   | -1    |    |



静态链表作为一种编程技巧，在有指针的编程语言中，也有广泛的应用



**例：**有若干个小球，从左至右依次编号为 $1, 2, 3, \dots, n$ 。可执行以下指令（保证 $X \neq Y$ ）：

- **L**  $X$   $Y$ 表示把小球 $X$ 移动到小球 $Y$ **左边**（如果 $X$ 已在 $Y$ 左边，则忽略该指令）。
- **R**  $X$   $Y$ 表示把小球 $X$ 移动到小球 $Y$ **右边**（如果 $X$ 已在 $Y$ 右边，则忽略该指令）。

例如 $n=6$ 时,初始:

1      2      3      4      5      6

执行指令L 1 5:

2      3      4      1      5      6

执行指令R 3 5:

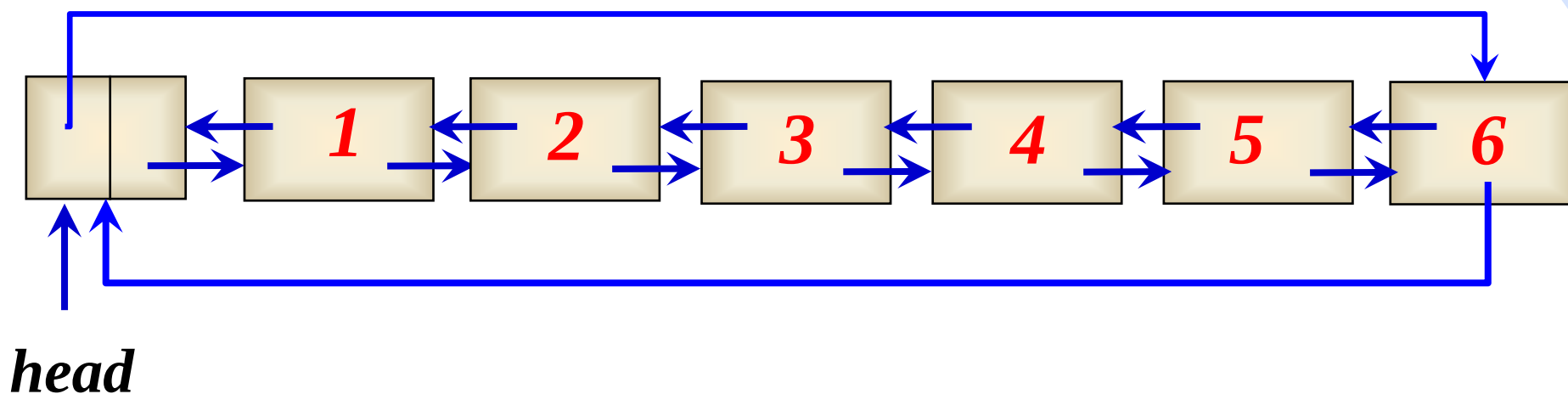
2      4      1      5      3      6

# 静态双向循环链表

```
int data[7];
```

data[x]: 第x号小球

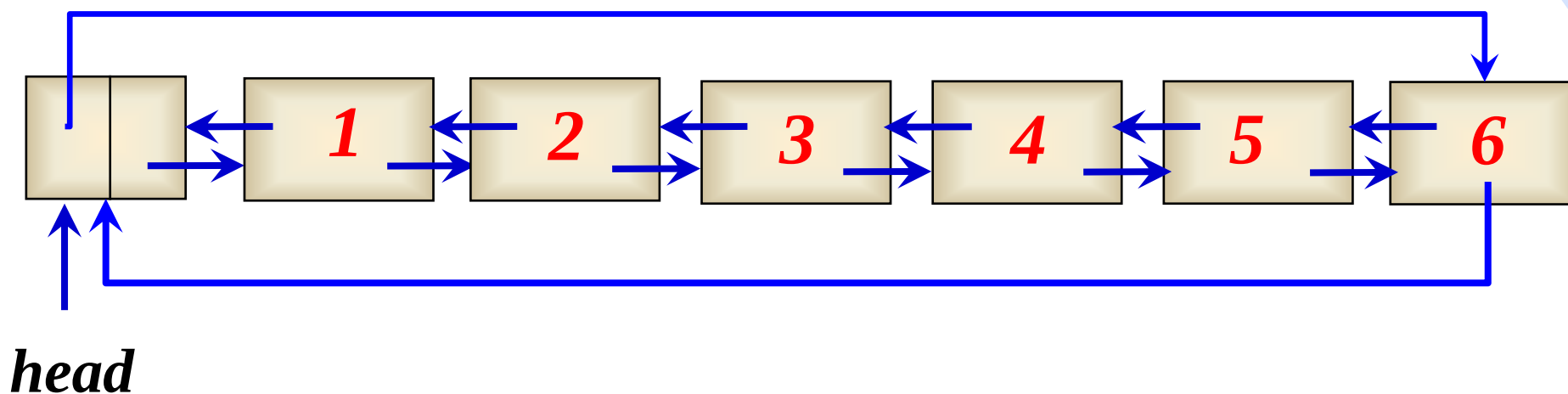
|      |  |   |   |   |   |   |   |
|------|--|---|---|---|---|---|---|
| data |  | 1 | 2 | 3 | 4 | 5 | 6 |
|------|--|---|---|---|---|---|---|



# 静态双向循环链表

```
int data[7];
int prev[7];
int next[7];
```

|      | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|------|---|---|---|---|---|---|---|
| prev |   |   |   |   |   |   |   |
| data |   | 1 | 2 | 3 | 4 | 5 | 6 |
| next |   |   |   |   |   |   |   |





# 静态双向循环链表

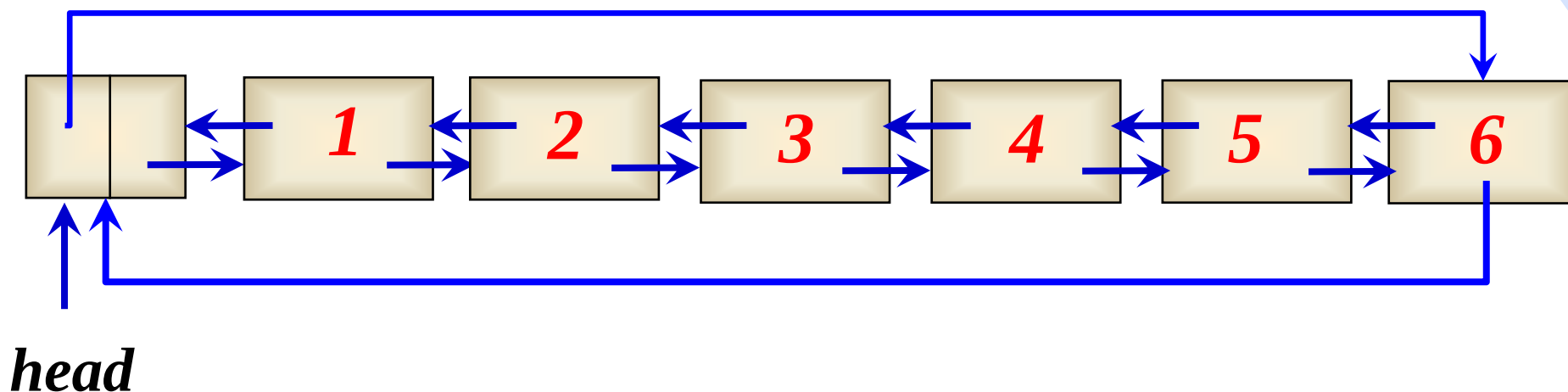
```
int data[7];
int prev[7];
int next[7];
```

x号球的左指针:  $prev[x]$   
x号球的右指针:  $next[x]$

在小球移动过程中仅  $prev$  和  $next$  “指针”变化。

|      | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|------|---|---|---|---|---|---|---|
| prev | 6 | 0 | 1 | 2 | 3 | 4 | 5 |
| data |   | 1 | 2 | 3 | 4 | 5 | 6 |
| next | 1 | 2 | 3 | 4 | 5 | 6 | 0 |

移动过程中小球在数组中的位置始终不变, 第  $i$  号球始终在数组第  $i$  位。所以可以在  $O(1)$  时间内找到任意小球。



删除小球x:

```
next[prev[x]]=next[x];
prev[next[x]]=prev[x];
```

小球x插到y右边:

```
prev[next[y]]=x;
next[x]=next[y];
prev[x]=y;
next[y]=x;
```

小球x插到y左边:

```
next[prev[y]]=x;
prev[x]=prev[y];
next[x]=y;
prev[y]=x;
```

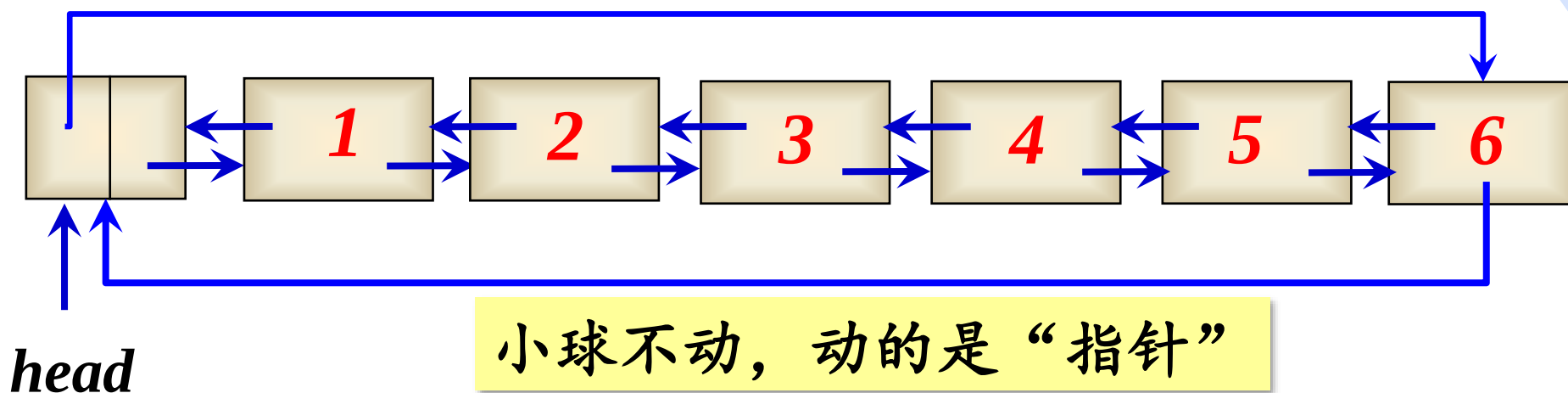
L X Y 操作:

- ①删除小球X
- ②将X插到Y左边

|      | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|------|---|---|---|---|---|---|---|
| prev | 6 | 0 | 1 | 2 | 3 | 4 | 5 |
| data |   | 1 | 2 | 3 | 4 | 5 | 6 |
| next | 1 | 2 | 3 | 4 | 5 | 6 | 0 |

R X Y 操作:

- ①删除小球X
- ②将X插到Y右边



可以省略  
data数组

兼顾数组  
链表优势



# 链表的高级主题选讲

- 静态链表
- **跳跃表**
- 侵入式链表

数据之法  
结构之美  
算法之道



# 线性结构的顺序查找

- 在包含 $n$ 个元素的数组 $R$ （假定下标从1开始）中查找值等于 $K$ 的元素。
- 若以无序方式维护这组元素，则应采用顺序查找。
- 从线性表的起始元素开始，逐个检查每个元素 $R_i$  ( $1 \leq i \leq n$ )，若查找成功，返回 $K$ 在 $R$ 中的下标，若查找失败，返回-1。

|   |   |   |    |   |   |    |   |    |
|---|---|---|----|---|---|----|---|----|
| 8 | 3 | 5 | 16 | 9 | 7 | 13 | 2 | 10 |
|---|---|---|----|---|---|----|---|----|

```
int Search(int R[], int n, int K){  
    for(int i=1; i<=n; i++)  
        if(R[i]==K) return i;  
    return -1;  
}
```

时间复杂度  
 $O(n)$

# 线性结构的对半查找

若以有序方式维护这组元素，则可采用对半查找。每次迭代将 $K$ 与当前子数组的中间元素比较，将查找范围缩小一半。

```
int BinarySearch(int R[],int n, int K){
```

```
//在数组R中对半查找K，R中元素递增有序
```

```
int low = 1, high = n, mid;
```

```
while(low <= high){
```

```
    mid=(low+high)/2;
```

```
    if(K<R[mid]) high=mid-1;
```

```
    else if(K>R[mid]) low=mid+1;
```

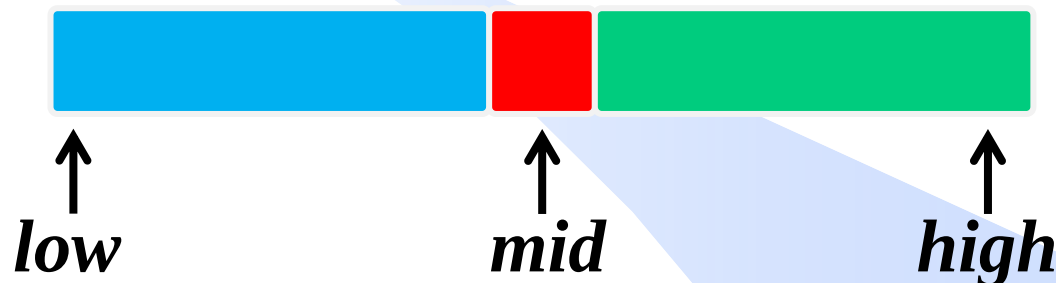
```
    else return mid;
```

```
}
```

```
return -1;
```

```
//查找失败
```

```
}
```



```
//在左半部分查找
```

```
//在右半部分查找
```

```
//查找成功
```

时间复杂度  
 $O(\log n)$

# 跳跃表 (*Skip List*) —— 动机

- 对半查找无法应用于链表。
- 能否借鉴对半查找的思想，提高有序链表中元素查找的效率？

## *Skip Lists: A Probabilistic Alternative to Balanced Trees*

William Pugh

Department of Computer Science, University of Maryland  
College Park, Maryland, USA, 20742  
email: pugh@mimsy.umd.edu

### Abstract

*Skip lists* are a practical, probabilistic data structure that can be used in place of balanced trees. Algorithms for insertion and deletion in skip lists are much simpler and significantly faster than equivalent algorithms for balanced trees. This paper describes and analyzes skip lists and presents new techniques for analyzing probabilistic algorithms.

### Introduction

Binary trees are used for representing abstract data types such as dictionaries and ordered lists, and work well when the elements are inserted in a random order. Some sequences of operations, such as inserting the elements in order, produce degenerate



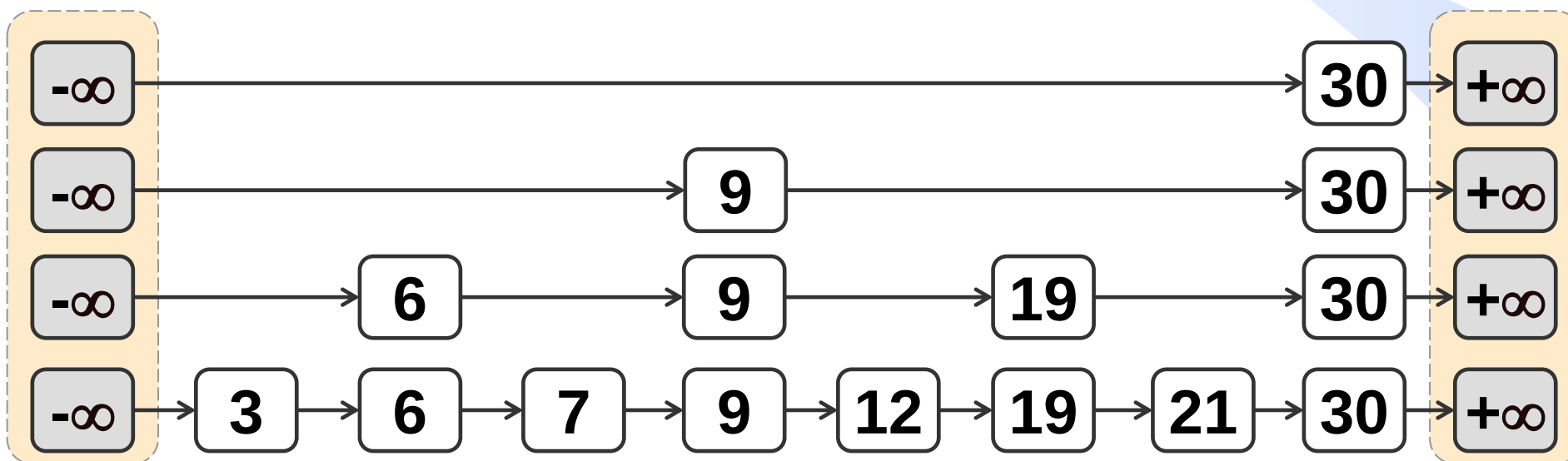
**William Pugh**  
康奈尔大学博士  
马里兰大学教授

William Pugh. *Skip Lists: A Probabilistic Alternative to Balanced Trees*. *Proceedings of Workshop on Algorithms and Data Structures*, 437-449, 1989.



# 跳跃表 (*Skip List*) —— 动机

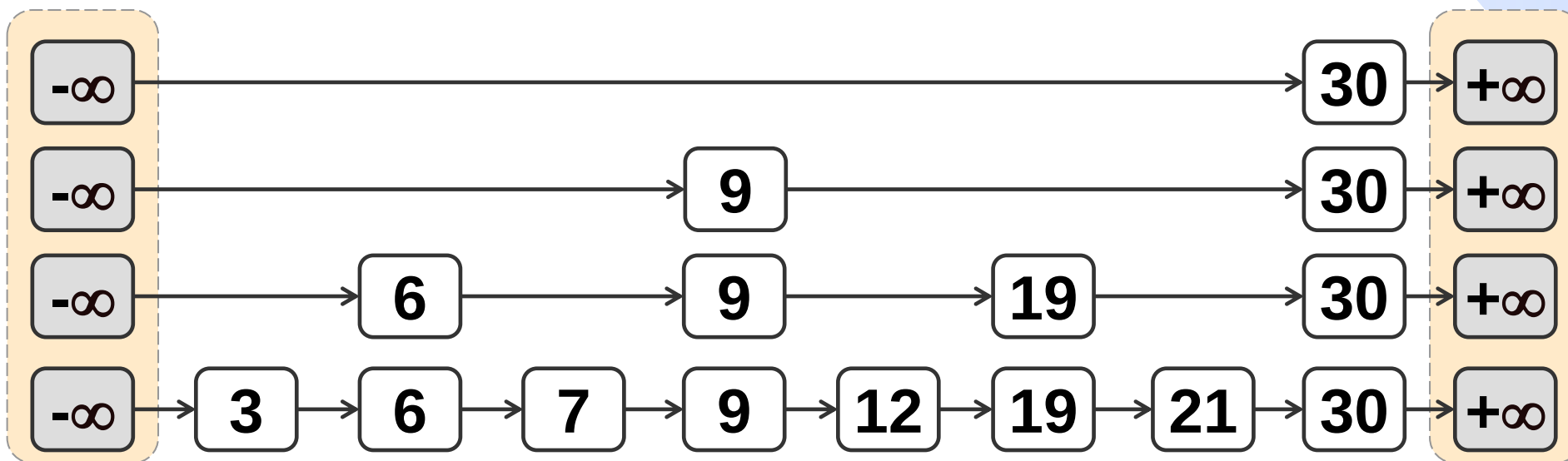
➤ 分层次、相互耦合的多个链表。



William Pugh. Skip Lists: A Probabilistic Alternative to Balanced Trees. *Proceedings of Workshop on Algorithms and Data Structures*, 437-449, 1989.

# 跳跃表 — 空间复杂度

- 自下而上的第0层有 $n$ 个结点
- 第1层期望结点个数:  $\lfloor n/2 \rfloor$
- 第2层期望结点个数:  $\lfloor n/4 \rfloor$
- 第3层期望结点个数:  $\lfloor n/8 \rfloor$
- 第 $i$ 层期望结点个数:  $\lfloor n/2^i \rfloor$



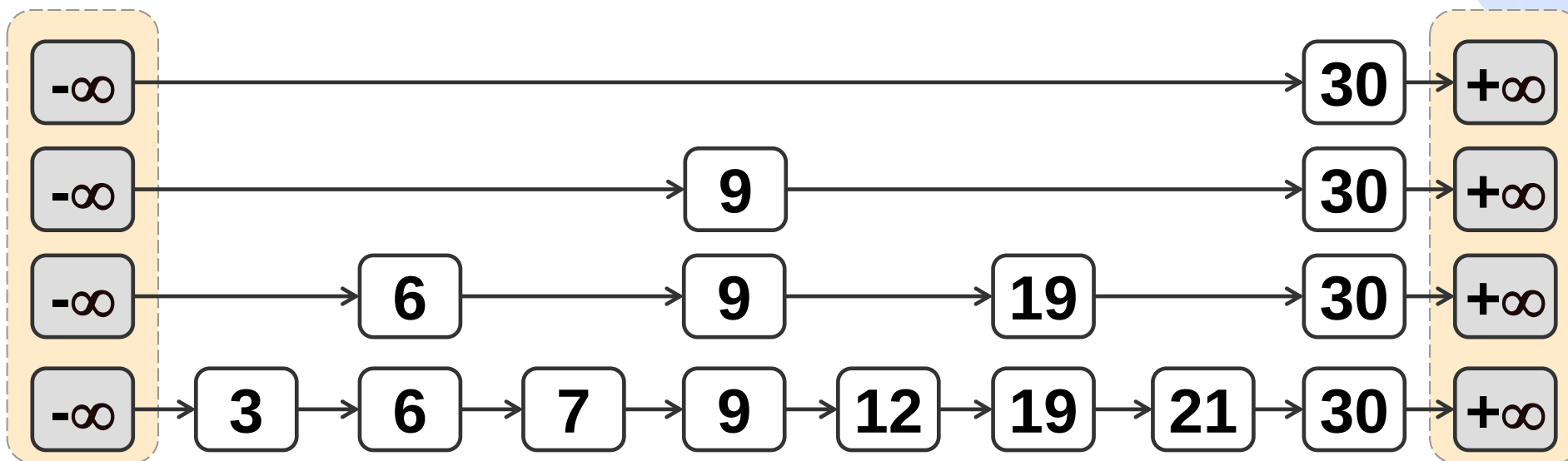


# 跳跃表 — 空间复杂度

➤ 第 $i$ 层期望结点个数:  $\lfloor n/2^i \rfloor$

➤ 假设有 $h$ 层, 则

$$\sum_{i=0}^h \left\lfloor \frac{n}{2^i} \right\rfloor < \sum_{i=0}^h \frac{n}{2^i} = n \sum_{i=0}^h \frac{1}{2^i} = 2n \left( 1 - \frac{1}{2^{h+1}} \right) < 2n = O(n)$$



# 跳跃表 — 查找

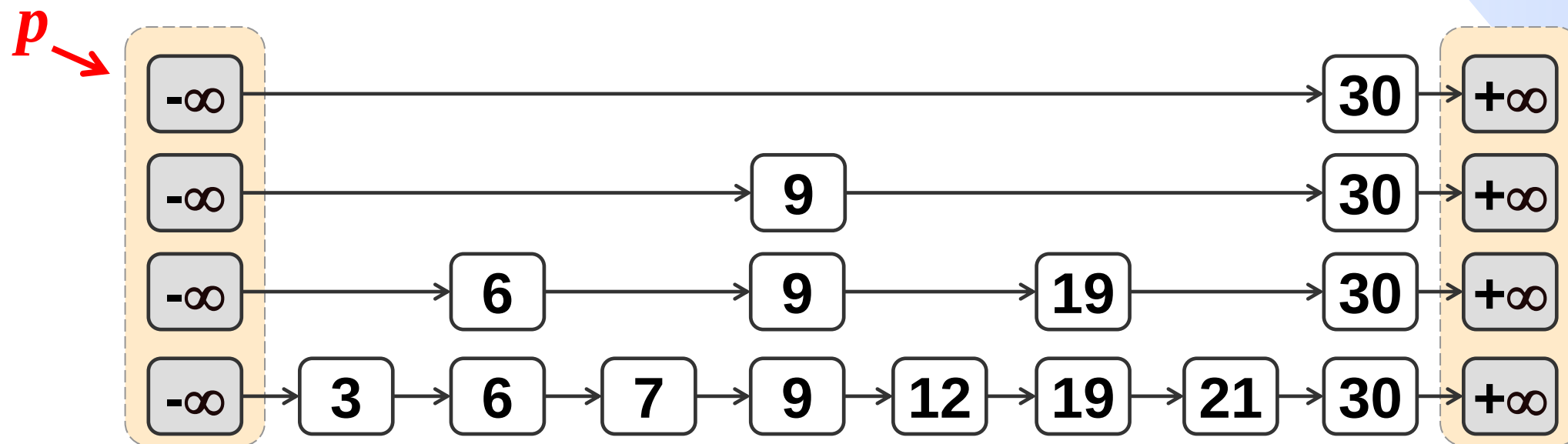
➤ 在跳表中查找元素 $K$  (例查找12)

令 $p$ 指向最上层第一个结点, 将 $K$ 与 $p \rightarrow next \rightarrow data$ 比较

①  $K < p \rightarrow next \rightarrow data$ :  $p$ 向下移动

②  $K > p \rightarrow next \rightarrow data$ :  $p$ 向右移动

③  $K = p \rightarrow next \rightarrow data$ : 查找成功



# 跳跃表 — 查找

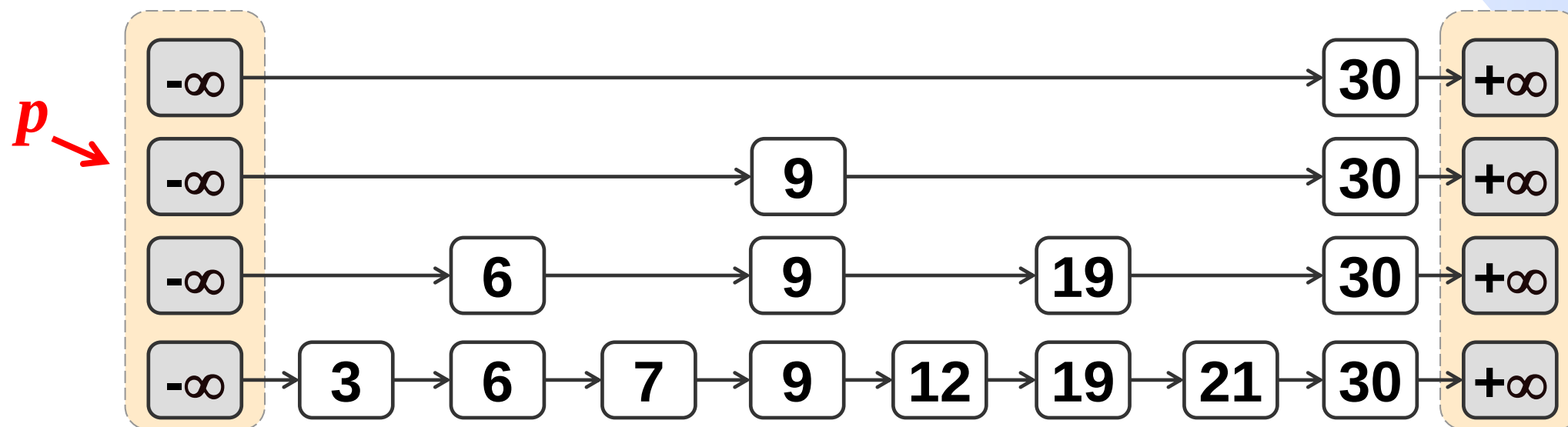
➤ 在跳表中查找元素  $K$  (例查找 12)

令  $p$  指向最上层第一个结点, 将  $K$  与  $p \rightarrow next \rightarrow data$  比较

①  $K < p \rightarrow next \rightarrow data$ :  $p$  向下移动

②  $K > p \rightarrow next \rightarrow data$ :  $p$  向右移动

③  $K = p \rightarrow next \rightarrow data$ : 查找成功



# 跳跃表 — 查找

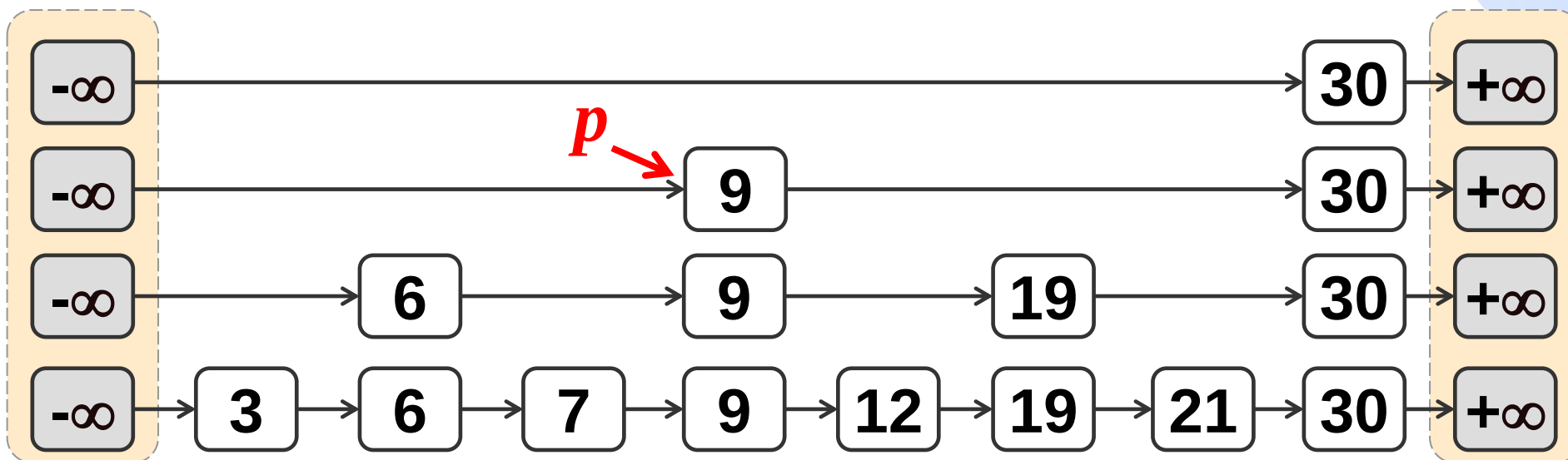
➤ 在跳表中查找元素 $K$  (例查找12)

令 $p$ 指向最上层第一个结点, 将 $K$ 与 $p \rightarrow next \rightarrow data$ 比较

①  $K < p \rightarrow next \rightarrow data$ :  $p$ 向下移动

②  $K > p \rightarrow next \rightarrow data$ :  $p$ 向右移动

③  $K = p \rightarrow next \rightarrow data$ : 查找成功



# 跳跃表 — 查找

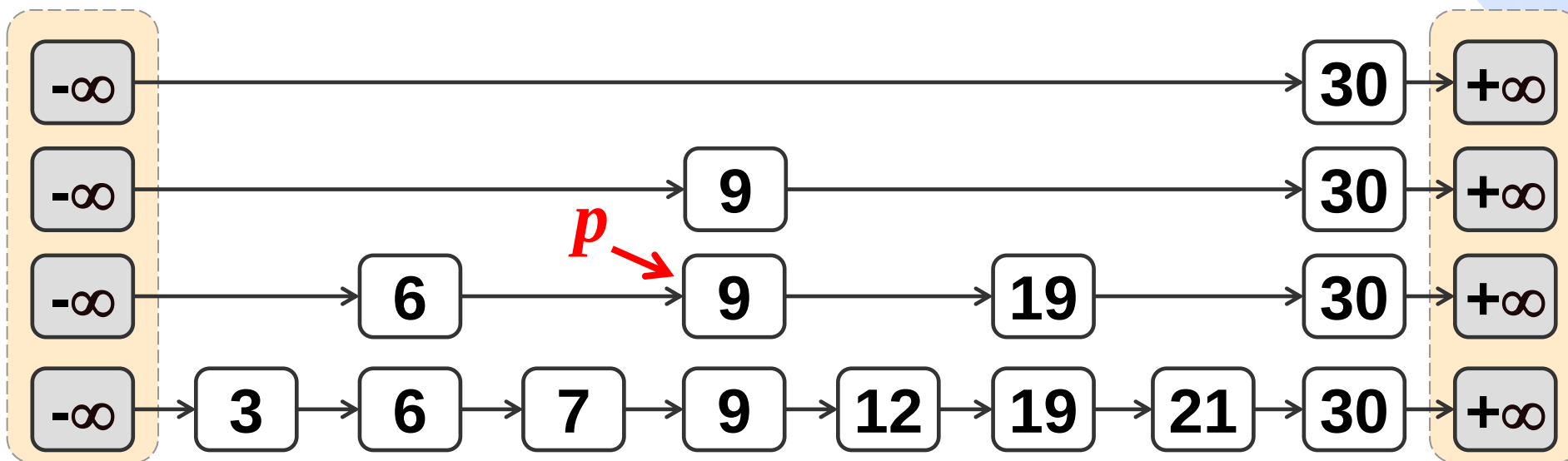
➤ 在跳表中查找元素 $K$  (例查找12)

令 $p$ 指向最上层第一个结点, 将 $K$ 与 $p \rightarrow next \rightarrow data$ 比较

①  $K < p \rightarrow next \rightarrow data$ :  $p$ 向下移动

②  $K > p \rightarrow next \rightarrow data$ :  $p$ 向右移动

③  $K = p \rightarrow next \rightarrow data$ : 查找成功



# 跳跃表 — 查找

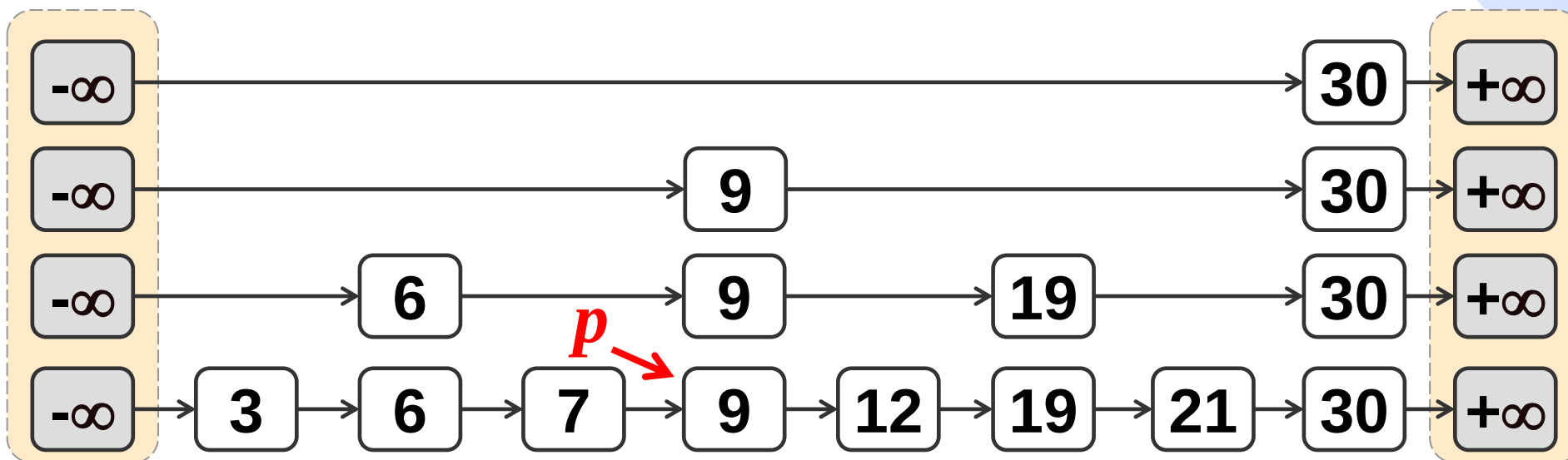
➤ 在跳表中查找元素 $K$  (例查找12)

令 $p$ 指向最上层第一个结点, 将 $K$ 与 $p \rightarrow next \rightarrow data$ 比较

①  $K < p \rightarrow next \rightarrow data$ :  $p$ 向下移动

②  $K > p \rightarrow next \rightarrow data$ :  $p$ 向右移动

③  $K = p \rightarrow next \rightarrow data$ : 查找成功



# 跳跃表 — 查找

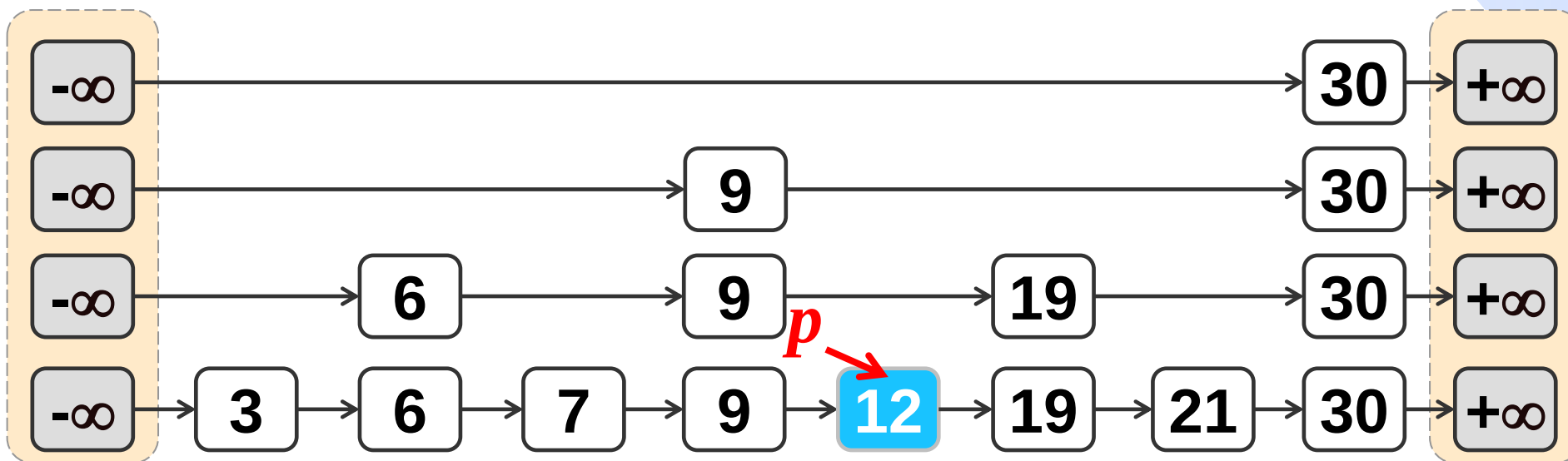
➤ 在跳表中查找元素 $K$  (例查找12)

令 $p$ 指向最上层第一个结点, 将 $K$ 与 $p \rightarrow next \rightarrow data$ 比较

①  $K < p \rightarrow next \rightarrow data$ :  $p$ 向下移动

②  $K > p \rightarrow next \rightarrow data$ :  $p$ 向右移动

③  $K = p \rightarrow next \rightarrow data$ : 查找成功

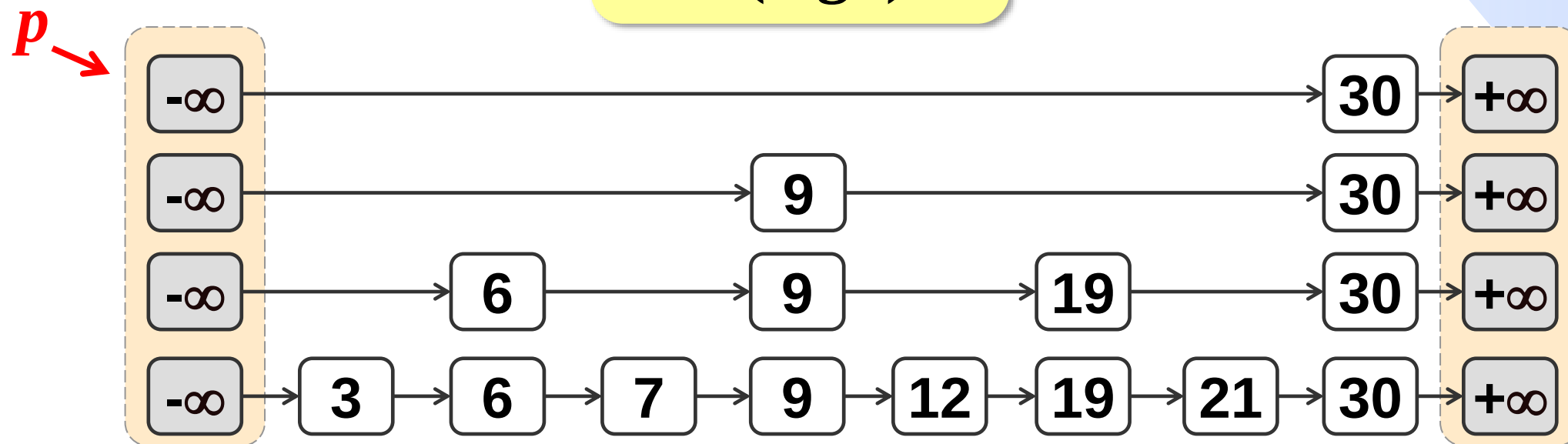




# 跳跃表 — 时间复杂度

- 跳跃表的层数  $h = O(\log n)$ ;
- 查找过程：由顶层到底层，每层元素比较次数为常数；
- 时间复杂度取决于跳跃表的层数。

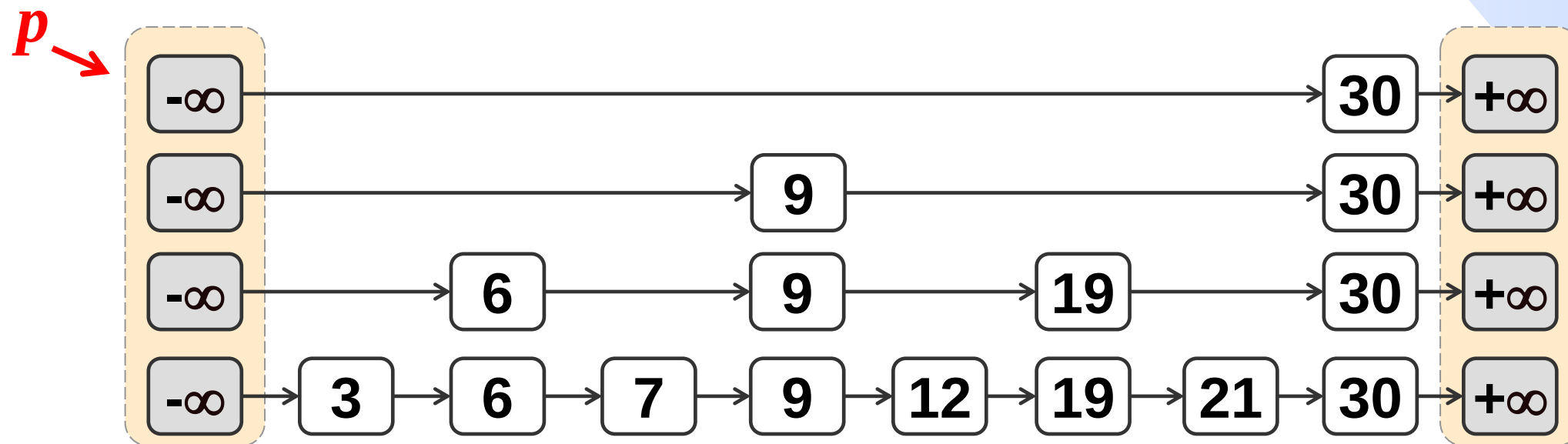
时间复杂度  
 $O(\log n)$



# 跳跃表 — 插入

➤ 在跳表中插入元素 $K$ （例插入17）

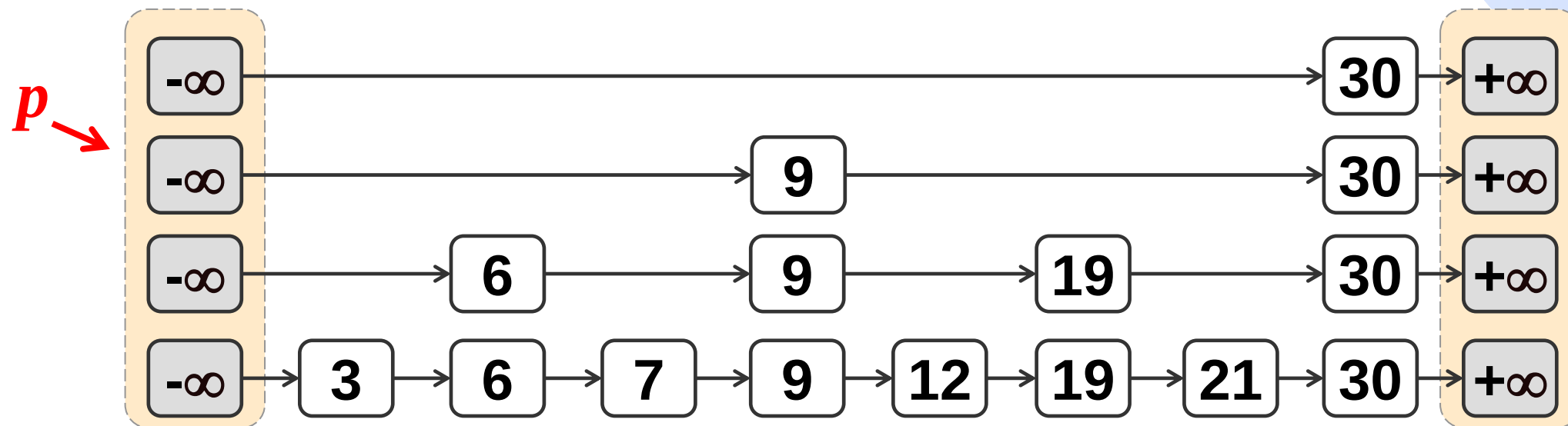
① 查找 $K$ ，在查找失败的位置 $p$ 插入 $K$ ；



# 跳跃表 — 插入

➤ 在跳表中插入元素  $K$ （例插入17）

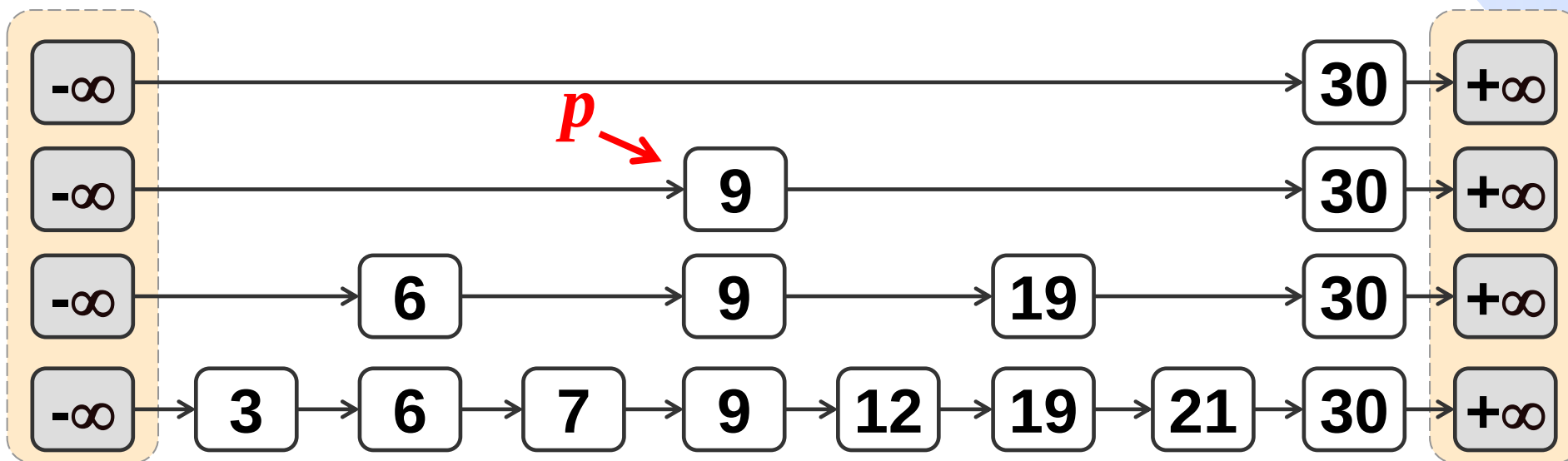
① 查找  $K$ ，在查找失败的位置  $p$  插入  $K$ ;



# 跳跃表 — 插入

➤ 在跳表中插入元素  $K$  (例插入17)

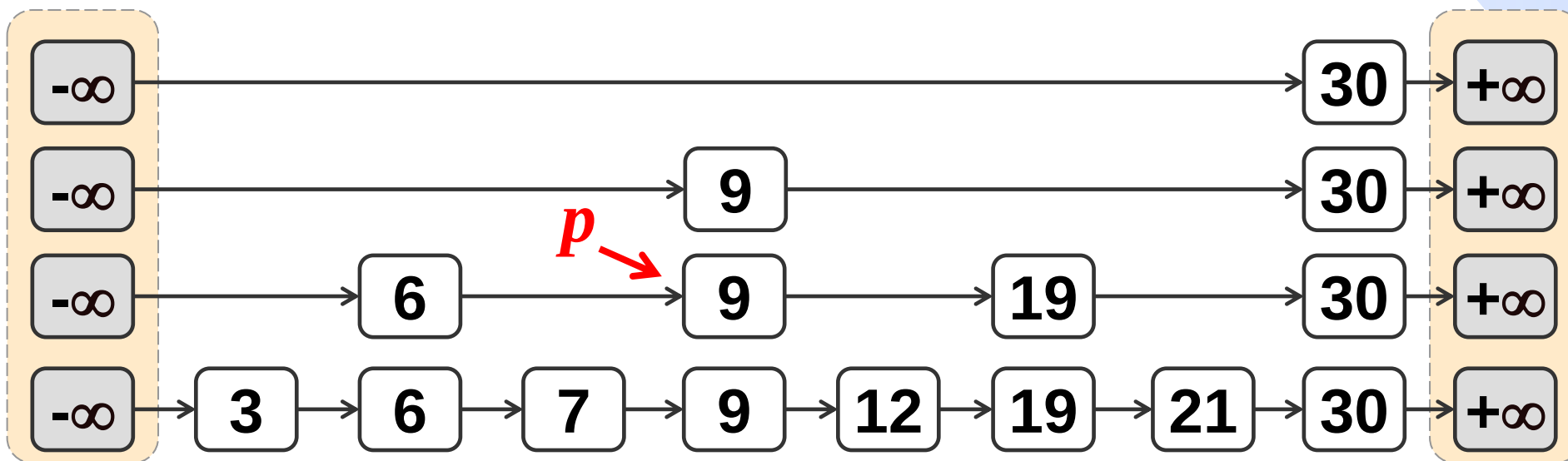
① 查找  $K$ ，在查找失败的位置  $p$  插入  $K$ ;



# 跳跃表 — 插入

➤ 在跳表中插入元素  $K$ （例插入17）

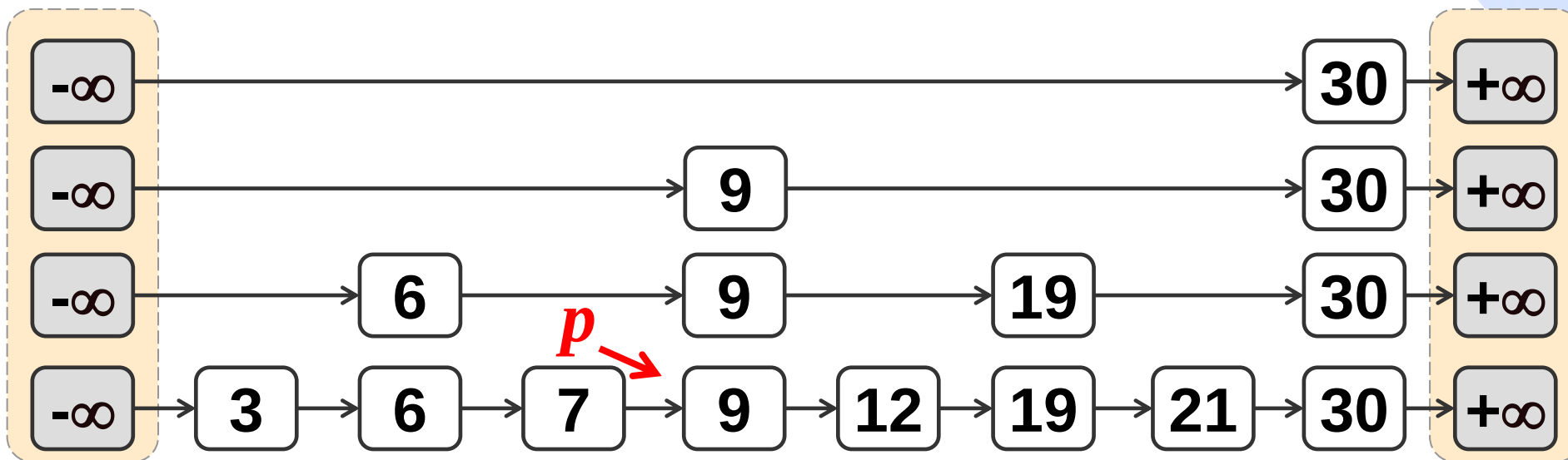
① 查找  $K$ ，在查找失败的位置  $p$  插入  $K$ ;



# 跳跃表 — 插入

➤ 在跳表中插入元素  $K$  (例插入17)

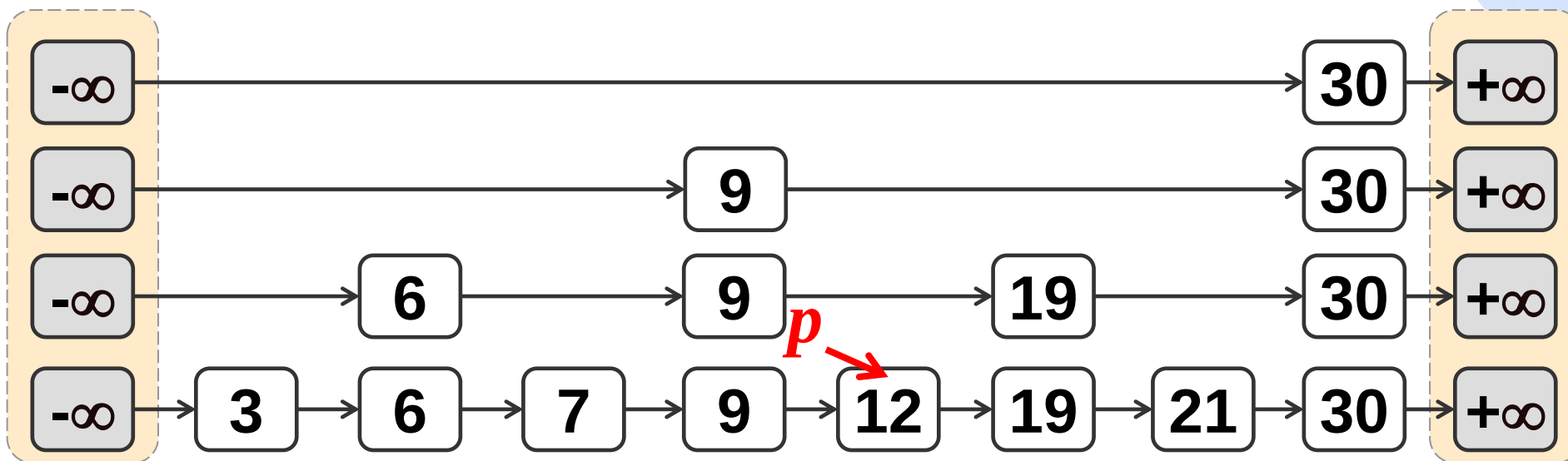
① 查找  $K$ ，在查找失败的位置  $p$  插入  $K$ ;



# 跳跃表 — 插入

➤ 在跳表中插入元素 $K$ （例插入17）

① 查找 $K$ ，在查找失败的位置 $p$ 插入 $K$ ；



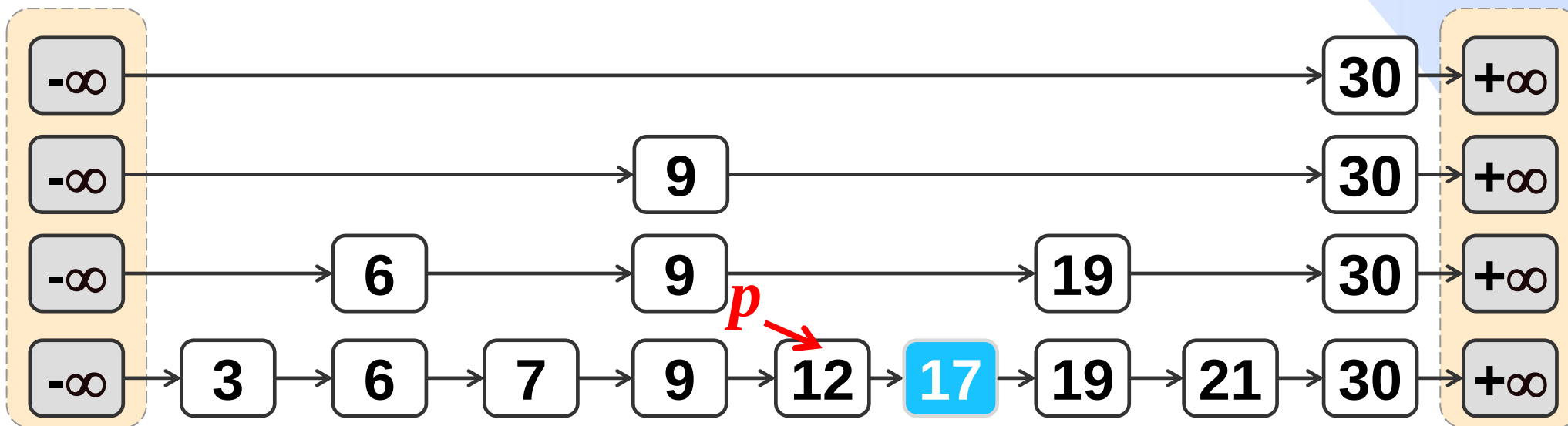
# 跳跃表 — 插入

➤ 在跳表中插入元素  $K$  (例插入 17)

`while(rand()%2==0)...`

① 查找  $K$ ，在查找失败的位置  $p$  插入  $K$ ;

② 以  $1/2$  的概率 (可通过抛硬币实现) 向上生长一层, 生长概率逐层减半。





# 跳跃表 — 插入

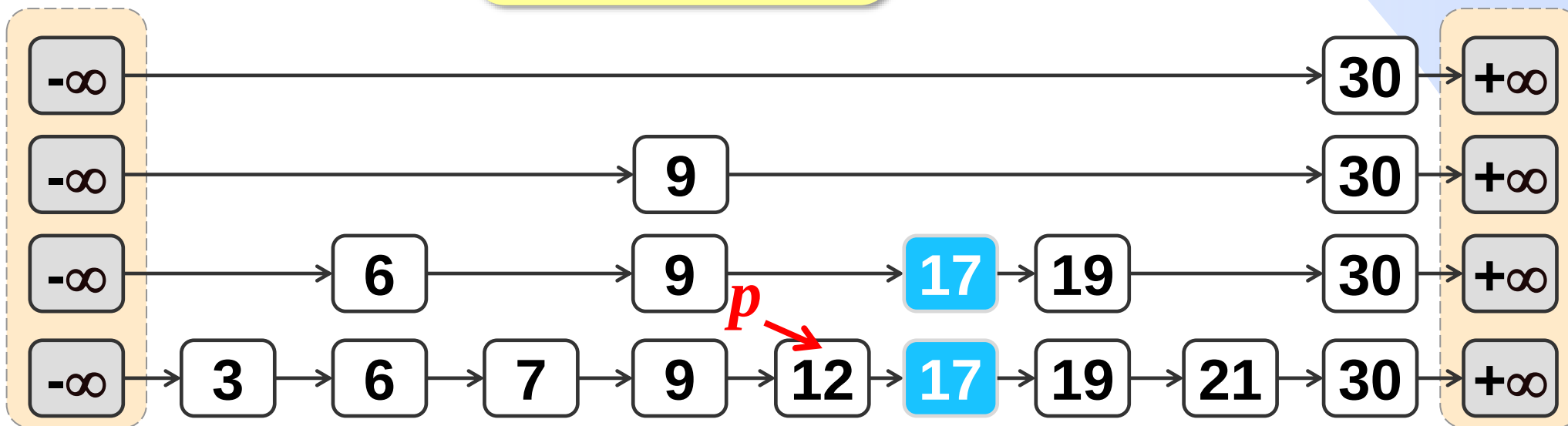
➤ 在跳表中插入元素 $K$ （例插入17）

`while(rand()%2==0)...`

① 查找 $K$ ，在查找失败的位置 $p$ 插入 $K$ ；

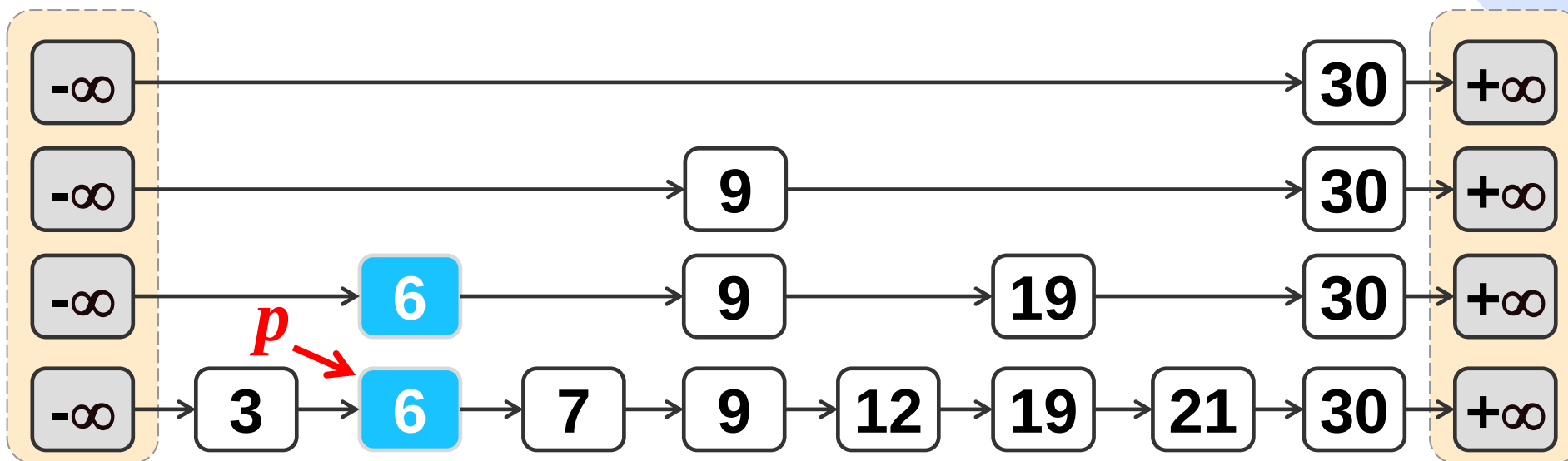
② 以 $1/2$ 的概率（可通过抛硬币实现）向上生长一层，**生长概率逐层减半。**

时间复杂度  
 $O(\log n)$



# 跳跃表 — 删除

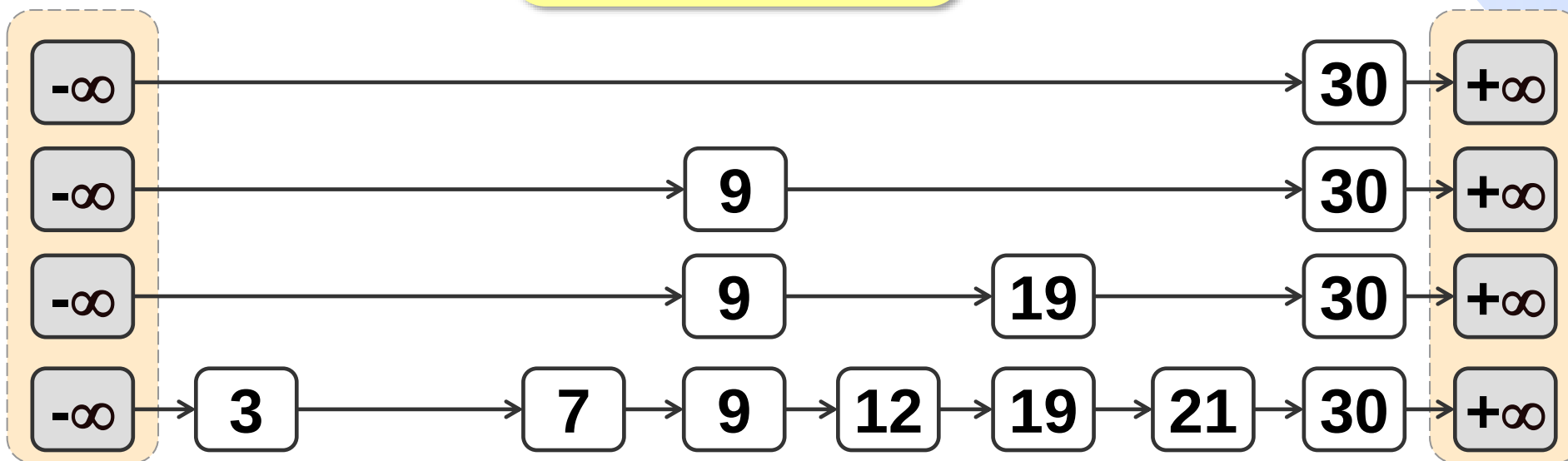
➤ 在跳表中删除元素 $K$ （例删除6）  
查找 $K$ ，删除每一层的 $K$ 。



# 跳跃表 — 删除

- 在跳表中删除元素 $K$ （例删除6）  
查找 $K$ ，删除每一层的 $K$ 。

时间复杂度  
 $O(\log n)$



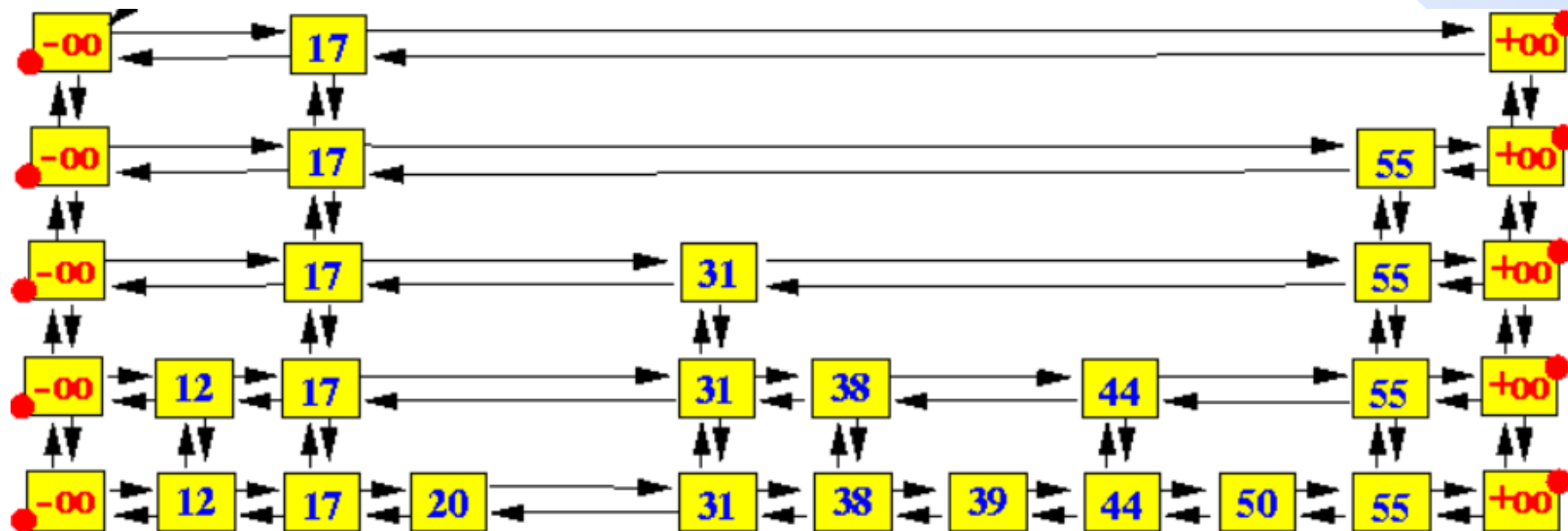
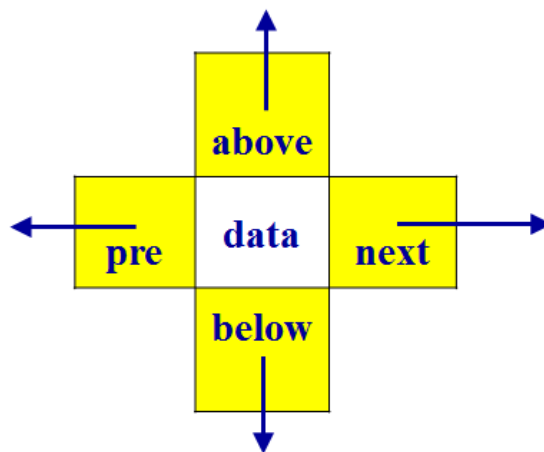


# 跳跃表 — 总结

- (1) 跳跃表的每一层都是一个有序的链表；
- (2) 最底层的链表包含所有元素；
- (3) 跳跃表是一种随机化的数据结构；
- (4) 跳跃表的查找、插入、删除时间复杂度为  $O(\log n)$ ；
- (5) 跳跃表的空间复杂度为  $O(n)$ ；
- (6) 适合动态查找场景。

# 跳跃表 — 实现

## ➤ 实现方式一：四联表

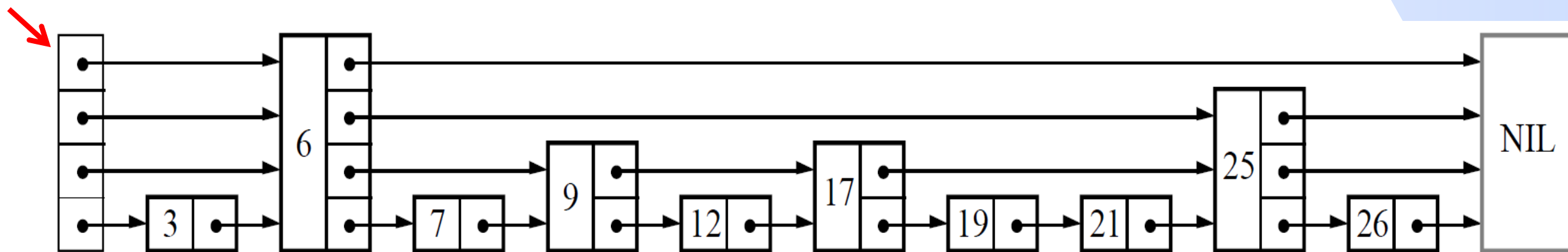


# 跳跃表 — 实现

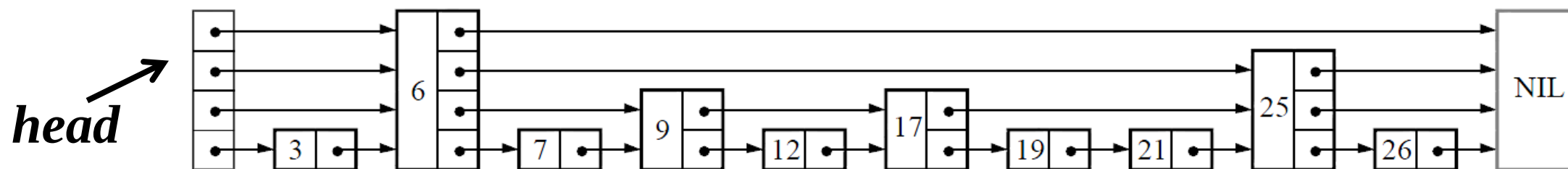
## ➤ 实现方式二：指针数组

```
#define MAXLEVEL 10
int level; //跳表的实际层数
struct SkipNode {
    int data;
    SkipNode* next[MAXLEVEL]={0};
    SkipNode(int d) {data = d;}
};
```

*head*



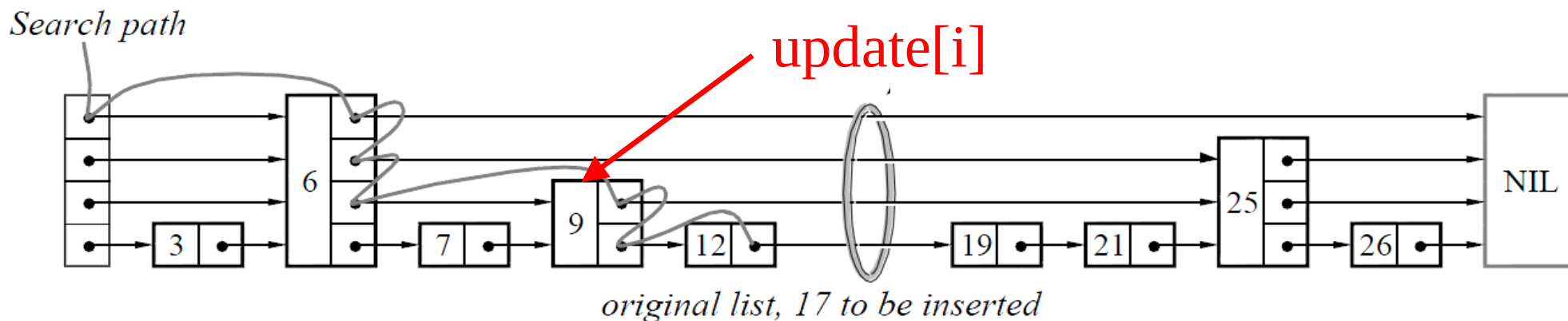
# 课下阅读：跳跃表—查找



```

SkipNode* search(SkipNode* head, int level, int K){
    //在头指针为head, 层数为level的跳表中查找值为K的结点, 返回指针
    SkipNode *p = head;
    for(int i=level; i>=0; i--){           //控制p下降
        while(p->next[i]!=NULL && p->next[i]->data<K) //控制p右移
            p = p->next[i];
    }
    p = p->next[0]; //p右移一位指向结点K
    if(p!=NULL && p->data == K) return p;
    return NULL;
}
    
```

# 课下阅读：跳跃表—插入



```
SkipNode* BuildUpdate(SkipNode* head, int level, int K, SkipNode *update[]){
```

```
//在表中查找K并构建update表
```

```
SkipNode *p = head;
```

```
for(int i=level-1; i>=0; i--) { //控制p下降
```

```
    while(p->next[i] != NULL && p->next[i]->data < K)
```

```
        p = p->next[i];
```

```
    update[i] = p;
```

```
}
```

```
p = p->next[0];
```

```
if(p != NULL && p->data == K)
```

```
    return p;
```

```
return NULL;
```

```
}
```

记录查找过程中每层（下降前）经过的最后一个结点存入update[]数组

```
int random_level() {
```

```
//以1/2概率向上生长，确定向上插入多少层
```

```
int newlevel = 1;
```

```
while(rand()%2==0 && newlevel < MAXLEVEL)
```

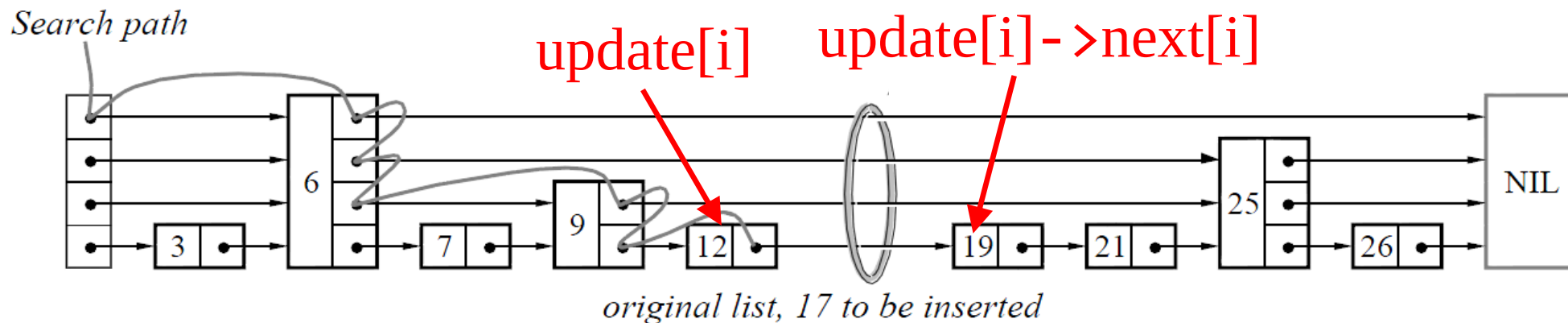
```
    newlevel++;
```

```
return newlevel;
```

```
}
```

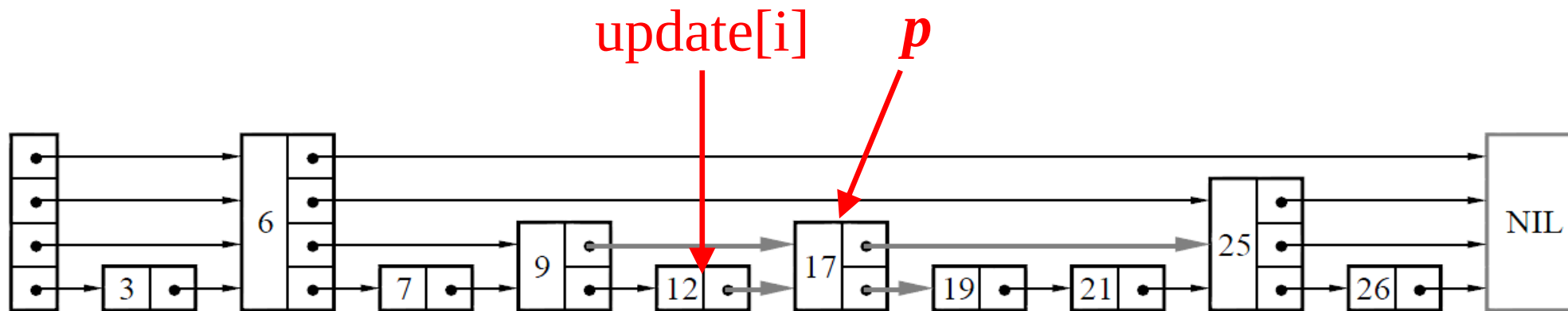


# 课下阅读：跳跃表—插入



```
void insert(SkipNode* head, int &level, int K) { //插入值为K的新结点
    SkipNode *update[MAXLEVEL] = { 0 };
    SkipNode *p = BuildUpdate(head, level, K, update); //在表中查找K并构建update表
    if(p != NULL) return; //表中已有K, 无需重复插入
    int newlevel = random_level(); //确定向上插几层
    if(newlevel > level) { //如果比原来层数还高, 更新level和update值
        for(int i = level; i < newlevel; i++)
            update[i] = head;
        level = newlevel;
    }
    p = new SkipNode(K); //创建新结点
    for(int i = 0; i < newlevel; i++) { //自下而上逐层插入结点p
        p->next[i] = update[i]->next[i];
        update[i]->next[i] = p;
    }
}
```

# 课下阅读：跳跃表—删除

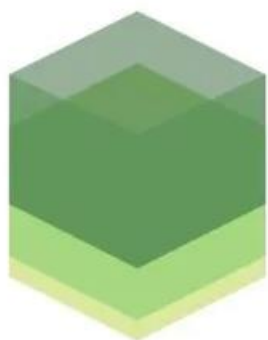


```
void remove(SkipNode* head, int &level, int K){ //删除值为K的结点
    SkipNode *update[MAXLEVEL] = { 0 };
    SkipNode *p = BuildUpdate(head, level, K, update); //在表中查找K并构建update表
    if(p == NULL) return; //表中未找到结点K, 无需删除
    for(int i = 0; i < level; i++) { //自底向上逐层拆下结点p
        if (update[i]->next[i] != p) break;
        update[i]->next[i] = p->next[i];
    }
    delete p;
    while(level>0 && head->next[level-1] == NULL) //删除后可能造成某些层变为空
        level--;
}
```

# 跳跃表应用



redis



LEVELDB





# 链表的高级主题选讲

- 静态链表
- 跳跃表
- **侵入式链表**

数据之法  
结构之美  
算法之道



# 普通链表（非侵入式）

```
struct Student{
    char name[20];
    double GPA;
    Student* next;
};
```

插入 InsertStudent(...)

删除 DeleteStudent(...)

```
struct Teacher{
    char name[20];
    char title[50];
    Teacher* next;
};
```

插入 InsertTeacher(...)

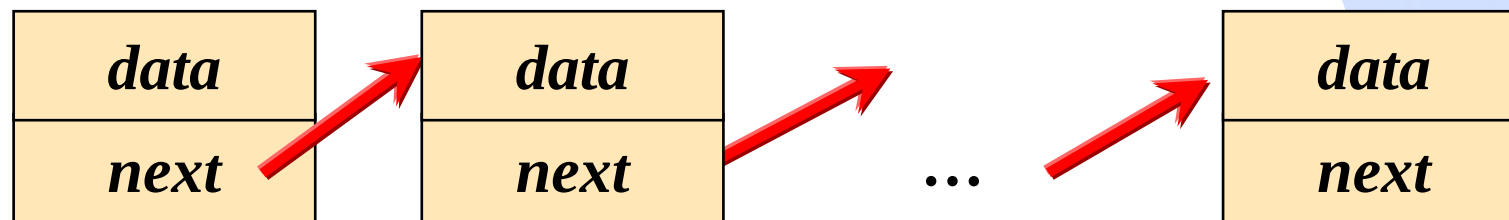
删除 DeleteTeacher(...)

```
struct Car{
    int ID;
    char owner[20];
    Car* next;
};
```

插入 InsertCar(...)

删除 DeleteCar(...)

数据与结构深度耦合，一种类型的链表只能应对一种业务场景，当业务场景改变之后，需要重新定义链表结构，重新实现链表的各种操作（如插入、删除、查找等）。



# 侵入式链表

一套基本的链表结构  
不含数据只含指针

```
struct ListPtr{
    ListPtr* next;
};
```

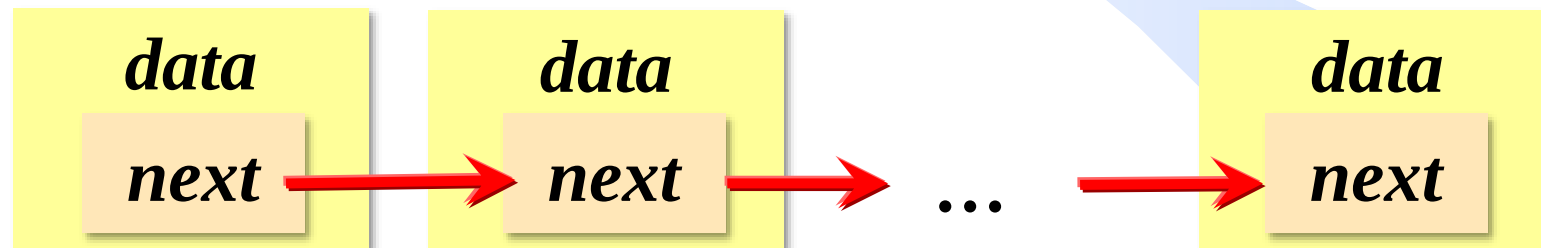
插入 InsertList(...)  
删除 DeleteList(...)

*next* → *next* → ... → *next*

```
struct Student{
    char name[20];
    double GPA;
    ListPtr list;
};
```

```
struct Teacher{
    char name[20];
    char title[50];
    ListPtr list;
};
```

```
struct Car{
    int number;
    int ower[20];
    ListPtr list;
};
```



将数据与结构解耦合。只需要定义一种链表结构 *ListPtr*，完成一套链表操作，即可实现对所有类型链表的统一管理。



# 侵入式链表的应用——操作系统内核链表

```
typedef struct _LIST_ENTRY {
    struct _LIST_ENTRY *Flink; // 前向指针
    struct _LIST_ENTRY *Blink; // 后向指针
} LIST_ENTRY;
```

Microsoft®  
**Windows**



```
struct list_head {
    struct list_head *next, *prev;
};
#define list_entry(ptr, type, member) \
    container_of(ptr, type, member)
#define container_of(ptr, type, member) ({ \
    const typeof(((type *)0)->member) *__mptr = (ptr); \
    (type *)((char *)__mptr - offsetof(type, member)); })

#define offsetof(TYPE, MEMBER) ((size_t) &((TYPE *)0)->MEMBER)

#define LIST_HEAD_INIT(name) { &(name), &(name) }
#define LIST_HEAD(name) \
    struct list_head name = LIST_HEAD_INIT(name)
```

Linux™



```
typedef struct LOS_DL_LIST { // 双向循环链表，鸿蒙内核最重要结构体之一
    struct LOS_DL_LIST *pstPrev; // 前驱结点
    struct LOS_DL_LIST *pstNext; // 后继结点
} LOS_DL_LIST;
// 将指定结点初始化为双向链表结点
LITE_OS_SEC_ALW_INLINE STATIC INLINE VOID LOS_ListInit(LOS_DL_LIST *list)
{
    list->pstNext = list;
    list->pstPrev = list;
}
// 将指定结点挂到双向链表头部
LITE_OS_SEC_ALW_INLINE STATIC INLINE VOID LOS_ListAdd(LOS_DL_LIST *list, LOS_DL_LIST *node)
{
    node->pstNext = list->pstNext;
    node->pstPrev = list;
    list->pstNext->pstPrev = node;
    list->pstNext = node;
}
```





# 自愿性质OJ练习题

- ✓ [LanqiaoOJ 1518](#)
- ✓ [LanqiaoOJ 1110](#)
- ✓ [LanqiaoOJ 1112](#)
- ✓ [LanqiaoOJ 1111](#)